Sarika S Pujari
AM132223

Python Foundations – Activity Handouts

Activity 0.1 – Everyday Task as an Algorithm

Title: From Daily Routine to Algorithm

Instructions to Students

1. Choose one simple daily activity (examples: making tea, preparing to go to class, booking a cab).
2. Write the steps in clear, numbered points.
3. Make sure each step is:
   - Unambiguous
   - In correct order
   - Does not skip important details

Template

1. Activity chosen: Microgreens production
2. Write the steps:
   1. Selection of good quality, viable seeds of crops like mustard
   2. Seed treatment : Clean seeds and soak them in water
   3. preparation of growing medium - use trays, cocopeat
   4. Sowing of seeds : evenly on the surface of the medium
   5. Watering of seeds regularly
   6. Harvesting - Harvest the seeds within 7-14 days
3. Check your algorithm:
   - Can a friend follow it without asking questions? Yes / No
   - If "No", refine the steps here:
     - Why we used to treat the seeds?

Why we select only good quality seed!

It is necessary to watering the seeds regularly

Activity 0.2 – From Steps to Pseudo-code

Title: Writing Pseudo-code with IF and REPEAT

Instructions

1. Take your steps from Activity 0.1.
2. Identify where decisions happen (IF/ELSE).
3. Identify where repetition happens (REPEAT / WHILE).
4. Rewrite your algorithm using the following keywords:
   - IF, ELSE, ELSE IF
   - REPEAT, WHILE, UNTIL

Template

1. Original activity: Microgreen production
2. Pseudo-code version:

text

START

Seed good quality seeds / viable seeds

IF growing medium is dry THEN

water gently

ELSE

Do not water

ENDIF pests or disease observed then take preventive measure

REPEAT

keep tray in shade, Maintain moisture

UNTIL seed germinate

END

3. Reflection: actions { If-medium is dry → water the tray
   else – Do not water

   - Where did you use IF/ELSE? ~~Repeat – keep tray in shade~~
   - Where did you use REPEAT/WHILE? keep tray in shade

---

* Monitoring and watering were continued while microgreens were not ready for harvest.

S Pujari

Activity 1.1 – Run and Edit Given Code

Title: First Python Script – Edit and Run

Instructions

1. Your teacher will give you a small Python script (example below).
2. Do NOT write new code first; only change text or numbers.
3. Run the program and observe the output.

Example Script

python

```
name = "Student"
age = 18
print("Hello,", name)
print("You are", age, "years old.")
```

Tasks

1. Change name and age to your own values.
   - name: "Sarika"
   - age: 22
2. Run the program. What is the output?

text

Output:

Hello, Sarika
You are 22 years old.

3. Change the printed message to:
   - "Welcome to Python, <your name>"

Write your edited code here:

python

 # Edited code:

4. Reflection:
   - What changed in the output and why?

Welcome to Python, Sarika

Activity 1.2 – Predict–Then–Run

Title: Predict Program Output

Instructions

1. Look at each code snippet.
2. BEFORE running, write what you think the output will be.
3. Then run it and compare.

Snippet A

python

```
x = 5
y = 2
z = x + y * 3
print(z)
```

z = x + y * 3
z = 5 + 2 * 3
z = 5 + 6
z = 11

- Predicted output: 11
- Actual output: 11

Snippet B

python

```
a = 10
b = a - 3
a = a + 1
print(a, b)
```

a = 10
b = a - 3
10 - 3 = $\boxed{7 = b}$
a = a + 1
10 + 1 = 11

- Predicted output: (11, 7)
- Actual output: (11, 7)

Reflection

- Where was your prediction wrong? Why?

Activity 2.1 – Calculator in Steps

Title: From Word Problem to Code

Problem 1
A student has marks in 3 subjects. Find total and average.

1. Write the steps in plain language:
   1. Take marks of 3 subject as input.
   2. Add all the marks to get total
   3. Divide total by 3 to get average marks
   4.
2. Now convert to Python (fill in the blanks):

python

mark1 = 80

mark2 = 95

mark3 = 85

total = 260

average = 86.6

print("Total =", total)

print("Average =", average)

total = mark1 + mark2 + mark3

Average = Total no. of marks / No. of subjects

$\frac{260}{3}$

print [average]

3. Test with your own numbers and record output.

Activity 2.2 – Fix the Errors

Title: Debugging Simple Python

Instructions
Each snippet has an error.

1. Write what you think is wrong.
2. Correct the code.

Snippet 1

python

print("Hello World)

- Error: In this sentence string is missing
- Corrected code: ("Hello world")

python

undefined

Snippet 2

python

a = 10

b = "5"

c = a + b

print(c)

- Error: Remove the string
- Corrected code: 5

python

undefined

Reflection:

- What did you learn about syntax vs logic errors?

Syntax errors – Occur when the rules of the programming language are violated (such as missing colons, these errors are detected by the compiler or interpreter. & prevent the program from running

• Logic error: occur when the program runs without errors but produces incorrect output due to wrong logic or incorrect steps in the program. These errors are not detected by the compiler and must be found by testing and debugging

Activity 3.1 – Rule-based Classification

Title: Grades with IF/ELSE

Rule

- If mark ≥ 90 : Grade A
- Else if mark ≥ 75 : Grade B
- Else : Grade C

Tasks

1. Draw the decision in simple text:
   - Condition 1: If mark greater than 90 grade A
   - Condition 2: If mark greater than 75 grade B
2. Now write Python code:

python

```
mark = int(input("Enter mark: "))
if mark ≥ 90 :
  print("Grade A")
elif mark ≥ 75 :
  print("Grade B")
else:
  print("Grade C")
```

3. Test with at least 3 marks and note outputs.

Input marks	output
marks-95	output - grade A
marks - 83	output - grade B
marks - 65	output - grade C

Activity 3.2 – Mini Problem Bank

Title: Structured Problem Solving with IF/ELSE

For each question, fill the template.

Template

1. Restate the problem in one sentence:

   To find the given number is even or odd.

2. Conditions needed (bullets):
   - if num % 2 == 0, print even number
   - else, print odd number

3. Python code:

python

```
# Your solution
num = int(input("Enter a number:"))
if num % 2 == 0
    print("even number")
else:
    print("odd number")
```

4. Test cases and results:
   - Input: 4 Output: even number
   - Input: 3 Output: odd number

Use this template for:

- Odd or even number.
- Ticket discount if age < 12 or > 60.
- Check if a year is a leap year (simple rule).

```
age = int(input("Enter age:"))
if age < 12
 print("Discount in ticket")
elif age > 60:
 print("discount in ticket")
else:
 print("No discount in ticket")
```

```
year = int(input("Enter a year:"))
if year % 4 == 0
    print("Leap year")
else:
    print("Not a leap year")
```

Activity 4.1 – Sum with Loops

Title: Sum from 1 to N

Steps

1. In plain language:
   - What is "sum from 1 to N"? Take the first number (1), add the next number (2), then the next (3) and keep adding until it reach n (n) 1+2+3+ --- n
2. Write the algorithm:

1. Start the program
2. Take a number n
3. Add numbers from 1 upto n
4. print the final sum
5. end the program

3. Now code it:

python

```
N = int(input("Enter N: "))
total = 0
for i in range( 1, n+1 ):
  total = total + i
print("Sum =", total)
```

4. Try N = 5, 10, 100 and check results.

---

		output
n = 5	→	15
n = 10	→	55
n = 100	→	5050

Activity 4.2 – Star Patterns

Title: Loops for Patterns

Pattern (triangle)

```
*
**
***
****
```

1. Explain the pattern in words: A triangle pattern in python is printed by using loops. for control rows and columns
2. Write code:

python

```
rows = int(input("Enter number of rows: "))
for i in range(1, rows + 1):
    print("*" * i, end=" ")
```

```
for
n=5
for i in range (1, n+1):
    for j in range (i):
        print ("*", end = " ")
    print ()
```

3. Try different rows and observe.

Activity 5.1 – Working with Lists

Title: Analyse Marks List

Given

python

marks = [56, 78, 90, 45, 88]

1. In words, write how to find:
   - Maximum mark: find maximum marks in the list
   - Average mark: find average marks in the list
   - Count of marks ≥ 60: find how many marks is greater than 60
2. Now write Python:

python

marks = [56, 78, 90, 45, 88]

Max

max_mark = 90

Average

total = 0

for m in marks:

total = 314 = total + m

average = 71.4 average = total / len (marks)

Count >= 60

count = 0 → count + 1

for m in marks:

if m >= 60

count = 78, 90, 88

print("Max:", max_mark)

print("Average:", average)

print("Count >= 60:", count)

Activity 5.2 – Choosing an Approach

Title: Which Method is Better?

Problem

Given a list of 1000 numbers, find how many are positive.

1. Approach A (loop) – describe in words:

Start a counter at 0, Look at the each number in the list, If the number is more than 0 add 1 to the counter, Do this for all numbers., the counter now shows how many numbers are positive

2. Approach B (using built-in functions or list comprehension) – describe:

Look at all numbers, keep only the positive ones, count them, that is the answer

3. Which is easier to understand for you now? Why?

Approach A is easier It shows step by step what is happening and each number is being checked

4. Implement one approach in Python:

python

numbers = [10, -3, 5, 0, 8, -2] # example

Your code

```
# Given list
numbers = [10, -3, 5, 0, 8, -2]
# Approach A: using loop
count = 0
for num in numbers:
    if num > 0:
        count += 1
print("positive numbers (loop): ", count)
```

output: positive number: 3,,

Activity 6.1 – Creating a Function

Title: Wrap Logic into a Function

Task

Turn average-of-marks logic into a function.

1. In words: what does the function do? : The average of mark function takes a list of marks as input, add all the marks together counts how many marks are there, and then divided the total by the counts to find the average marks. It returns average value

2. Write code:

python

```
def average_marks(marks_list):
  total = 0
  for m in marks_list:
    total = total + mark
  avg = total / len (marks)
  return avg

data = [60, 70, 80]
print("Average =", average_marks(data))
```

3. Test with another list and note output.

Activity 6.2 – Problem-Solving Template

Title: Standard Template for Any Problem

Use this for any new problem.

1. Problem understanding
   - What is given (inputs)?

Base fee of the class, age of the student.

   - What is required (outputs)?

The final fee to be paid after applying the applicable discounts

   - Any constraints (limits, ranges)?

Age should be a positive integer, Base fees should be positive number, only one discount can be applied. discount applies only if Age < 12 Age > 60

2. Analysis
   - Write steps in bullets (algorithm):
     - Start, Read the base fee, Read the age of student
     - Check if age is less than 12. if yes, calculate 50% discount & reduce it from the base fee
     - Else check if age is greater than 60. If yes, calculate 30% discount & reduce it from the base fees. otherwise do not apply any discount. Display the final fee to be paid. Stop
3. Design
   - Will you use loops? Which? No
   - Will you use conditions (if/else)? yes
   - Will you use functions? Which ones? yes, user defined function
   - What data structures (list, etc.)? No
4. Code (Python) :

```
fee = 1000
Age = 10
if age < 12:
    fee = fee - (fee * 0.50)
elif age > 60:
    fee = fee - (fee * 0.30)
print (fee)
```

python

 # Your implementation

5. Test
   - Test case 1: input 10, 1000 → output 500
   - Test case 2: input 65, 5000 → output 3500
   - Edge case: input 45, 4000 → output 4000

Activity 7.1 – Capstone Problem

Title: Simple Fee Calculator

Problem

A class charges a base fee.

- If student is below 12 years, 50% discount.
- If above 60, 30% discount.
- Otherwise, no discount.

Use the full template from Activity 6.2 here.

(Attach a blank copy of the template and ask students to fill all five sections.)

Activity 7.2 – Reflection Sheet

Title: How I Solve a New Problem

1. When I see a new problem, my first step is:

my first step is to read it carefully & understand what is required

2. Before coding, I make sure I know:

   - Inputs: what data is given
   - Outputs: what result is expected

3. The tools I think about (loops, if, functions) are:

It based on the problem requirements and the logic needed solve it

4. One thing I will do differently from now on:

plan the logic before writing the code